
ROBUST REINFORCEMENT LEARNING AGAINST SPURIOUS CORRELATIONS

November 2nd, 2025

Zahra Khodabakhshian
Mtrk.nr.: 426198
Rheinland-Pfälzische Technische Universität Kaiserslautern-Landau (RPTU)
jov98mam@rptu.de

1. Motivation

Reinforcement learning (RL) agents can perform very well in the environment in which they are trained, but this performance does not always transfer to slightly different settings. One common reason is that the agent may learn a shortcut from the training data. In other words, it may rely on a feature that is correlated with success during training, although this feature is not actually needed for solving the task.

This problem is the main motivation for my thesis. In a robotic manipulation task, for example, the color of an object can be correlated with its position. The robot may then learn to use color as a cue, even though color is not what makes the lifting task succeed. If this relationship changes later, the learned policy may fail. The aim of this work is to study this kind of failure in a controlled RL setting and to investigate whether training with generated counterfactual transitions can make the policy less dependent on such misleading correlations.

2. Research Direction

The project is inspired by recent work on robust reinforcement learning against spurious correlations, especially the RSC-MDP framework from *Seeing is not Believing* [1]. The main idea is that some observed state features may be correlated because of an unobserved confounder. If the distribution of this hidden confounder changes, the correlation between observed features can also change. A policy that relies on the spurious feature may then fail, while a policy that relies on the real causal structure of the task should remain robust. The paper studies cases where spurious correlations are caused by hidden confounders and proposes to generate additional transitions that break or weaken these correlations.

My implementation adapts this idea to robosuite Lift. I create a training environment where cube color and cube position are correlated, and then I evaluate the trained policy in shifted environments where this relationship is changed or removed. The goal is to test whether the RSC-SAC idea can reduce the policy's dependence on the spurious color feature and improve generalization under distribution shift.

3. Proposed Method

My implementation is an RSC-SAC system for the robosuite Lift task. The method starts by creating a modified Lift environment in which a designed spurious correlation is introduced between the cube position and the cube color. The cube RGB color is then appended to the original observation vector, so the agent can observe both the physical state of the robot/cube and the potentially spurious color feature.

The agent is trained with Soft Actor-Critic (SAC), but during training the standard SAC replay-buffer update is modified using the RSC procedure from the paper. First, a batch of real transitions is sampled from the replay buffer. Then, a fraction β of the samples is selected for RSC intervention. For these selected samples, the current state is perturbed using Eq. 7. After perturbation, a D.1-style causal transition model predicts the corresponding next state and reward. Finally, part of the SAC batch is replaced with these synthetic transitions, and SAC is updated on the mixed real/synthetic batch.

The full training and evaluation flow is:

1. Create the robosuite Lift environment.
2. Add a designed spurious correlation between cube position and cube color.
3. Append the cube RGB color to the observation vector.
4. Collect real transitions using SAC and store them in the replay buffer.
5. Sample a batch of real transitions from the replay buffer.
6. Select β percent of the batch for RSC perturbation.
7. Perturb the selected current states using Eq. 7.
8. Train the D.1-style causal transition model on the selected samples.
9. Use the transition model to predict synthetic next state and reward for the perturbed states.
10. Replace the selected rows in the SAC batch with these synthetic transitions.
11. Update SAC using the mixed real/synthetic batch.
12. Evaluate the learned policy on different spurious test modes.

3.1. Environment Design

The environment is based on robosuite Lift with a Panda robot. The original task is to lift a cube from the table. In the standard task, the agent should rely on physical information such as the cube position, gripper position, and robot state.

To study spurious correlation, I modify the environment so that cube color becomes correlated with cube position during training. For example, one cube position may usually correspond to green and the other to red. The RGB value of the cube is appended to the state observation. This means the agent can observe a feature that is useful during training but not necessarily causal for solving the task.

The environment supports different spurious modes:

- `confounded`: color and position are correlated. This is the nominal training setting.
- `shifted_indep`: color and position are independent. This tests whether the policy still works when the shortcut is removed.
- `shifted_swapped`: the color-position mapping is swapped. This tests whether the policy fails when the shortcut becomes misleading.

3.2. Base RL Algorithm: SAC

The base reinforcement learning algorithm is Soft Actor-Critic. SAC learns a stochastic actor policy and two critic networks using transitions sampled from a replay buffer. This provides the standard baseline.

The RSC method is not a replacement for SAC. Instead, it modifies the data used during SAC updates. The policy still learns with the normal SAC actor-critic objective, but part of each training batch is replaced with RSC-generated synthetic transitions.

3.3. Eq. 7 State Perturbation

For selected samples in a batch, Eq. 7 is used to perturb the current state. The idea is to choose one state dimension and replace it with the value from another sample in the same batch. The partner sample is chosen so that it is very different in the selected dimension but similar in the remaining dimensions.

$$s_t^i \leftarrow s_k^i, \quad k = \arg \max \frac{\|s_t^i - s_k^i\|_2^2}{\sum_{\tilde{i}} \|s_t^{\tilde{i}} - s_k^{\tilde{i}}\|_2^2}, \quad k \in \{1, \dots, K\} \quad (7)$$

Intuitively, this creates a counterfactual-like state. One feature is changed while the rest of the state remains close to the original sample. In the context of Lift, this can help break the relationship between cube color and cube position.

3.4. Causal Transition Model

After perturbing the current state, the original next state and reward may no longer match the modified state. Therefore, the method uses a learned transition model to predict the next state and reward for the perturbed state.

The transition model follows the structure from Appendix D.1 of the paper. It treats each state and action dimension as a scalar variable. Each scalar is encoded with a shared encoder and a learnable position embedding. A learnable causal graph is then applied between current state/action variables and next-state/reward variables. Finally, a shared decoder predicts the next state and reward.

3.5. Mixed Real/Synthetic SAC Update

The final SAC update is performed on a mixed batch. Most samples remain real replay-buffer transitions. The selected RSC samples are replaced by synthetic transitions consisting of:

- perturbed current state,
- original action,
- predicted next state,
- predicted reward.

This exposes the policy and critic to transitions where the spurious feature has been changed. The goal is to make the learned policy less dependent on cube color and more dependent on physical task-relevant information.

4. Expected Effect

The expected effect of the method is that the learned policy becomes less dependent on the cube-color shortcut and more dependent on task-relevant physical information, such as the cube position, gripper position, and robot state.

A standard SAC policy may achieve high performance in the confounded training environment because the color-position shortcut is useful there. However, when the color-position relationship changes at test time, this shortcut can become unreliable or even misleading. In that case, the policy may fail because it has learned a spurious correlation instead of a robust lifting strategy.

In contrast, the RSC-SAC approach is expected to improve robustness by training the agent on perturbed, counterfactual-like transitions. These transitions weaken the original spurious correlation and expose the policy to states where the shortcut is less reliable. Ideally, this encourages the policy to rely more on causal and task-relevant features rather than cube color alone.

5. Research Questions

This thesis investigates the following research questions:

1. Does a standard SAC agent trained in the confounded Lift environment rely on the spurious color-position correlation?
2. Can RSC-style counterfactual transition generation improve robustness under shifted test environments?
3. How important is the perturbation strategy, especially when comparing paper-faithful random perturbation with color-focused perturbation?

6. Expected Outcome

The expected outcome is an empirical study of robust RL under a controlled spurious correlation. I expect to compare standard SAC with an RSC-inspired variant that uses generated transitions. The main evaluation will measure success rates in the original confounded environment and in shifted environments where the spurious relationship changes.

The contribution of the thesis will be a clear experimental analysis rather than a claim that one method solves the full problem. If the robust method improves performance under shift, the thesis can show how counterfactual transition generation helps reduce shortcut learning. If the improvement is limited, the work can still be useful by showing where the method becomes unstable or where the perturbation design matters. In both cases, the goal is to better understand how RL agents behave when misleading correlations are present in the training environment.

6.1. Preliminary Results

The following tables show preliminary results from experiments comparing a baseline SAC agent (Base) with an RSC-SAC agent using random perturbation (Random):

Table 1: Testing reward on shifted environments

Environment	Base (SAC)	Random (RSC-SAC)
Shifted independent	0.412	0.647
Shifted swapped	0.000	0.312

Table 2: Testing reward on nominal environments.

Environment	Base (SAC)	Random (RSC-SAC)
Confounded (nominal)	1.000	0.824

Note on Metrics: The paper reports normalized rewards, while I report raw episodic returns. My raw return is the sum of shaped robosuite rewards over an episode. The paper divides each method’s mean episode return by vanilla SAC’s mean episode return in the nominal environment. Because I have not applied this normalization yet, my numbers show performance on my own reward scale and are mainly comparable within my experiments, not directly against the paper tables.

References

- [1] Wenhao Ding and Laixi Shi and Yuejie Chi and Ding Zhao, “Seeing is not Believing: Robust Reinforcement Learning against Spurious Correlation.”